

Main.py

```
1  ## MAIN BEAM ELEMENT DOCUMENT ##
2
3  # Import packages
4  import sys
5  print(sys.version, flush=True)
6  import time
7  from ansys.mapdl.core import launch_mapdl
8
9  # Import Functions
10 import optimization
11 from MyAPDLCall import RunAPDL
12
13 # Variables. Choose which ones to include by setting "active": True or False.
14 opti_settings = {
15     "n_mast_segments": 5,          # Number of mast segments
16     "mast_segment_height": 810,    # Height of each mast segment [mm]
17     "segment_mass_limit": 23,      # Limits for segment masses [kg]
18     "multi_size_columns": True,    # Whether mast segments columns uses different dimensions (True) or not (False)
19     "multi_size_braces": True,     # Whether mast segments braces uses different dimensions (True) or not (False)
20     "brace_split": True,          # Whether braces are split between horizontal and cross (True) or not (False)
21 }
22
23 # Initial Guess
24 column_diameter = 76.1 # Column Diameter [mm]
25 column_thickness = 3 # Column Thickness [mm]
26 brace_diameter = 26.9 # Brace Diameter [mm]
27 brace_thickness = 2.3 # Brace Thickness [mm]
28
29
30 # Bounds
31 column_diameter_bounds = (48.3, 114.3) # Column Diameter Bounds [mm]
32 column_thickness_bounds = (2.5, 5.0) # Column Thickness Bounds [mm]
33 brace_diameter_bounds = (10.0, 50.0) # Brace Diameter Bounds [mm]
34 brace_thickness_bounds = (1.0, 4.0) # Brace Thickness Bounds [mm]
35
36 # Defining variables with bounds and active status
37 var = {
38     "rad": {"value": 202.07, "bounds": (150.0, 350.0), "active": True}, # Radius Structure [mm]
39 }
40 if opti_settings["multi_size_columns"]:
41     var.update({
42         **{"d0_{i}": {"value": column_diameter, "bounds": column_diameter_bounds, "active": True} for i in range(1, opti_settings["n_mast_segments"]+1)},
43         **{"t0_{i}": {"value": column_thickness, "bounds": column_thickness_bounds, "active": True} for i in range(1, opti_settings["n_mast_segments"]+1)},
44     })
45 else:
46     var.update({
47         "d0": {"value": column_diameter, "bounds": column_diameter_bounds, "active": True}, # Column Diameter [mm]
48         "t0": {"value": column_thickness, "bounds": column_thickness_bounds, "active": True}, # Column Thickness [mm]
49     })
50 if opti_settings["multi_size_braces"]:
51     if opti_settings["brace_split"]:
52         var.update({
53             **{"d1_h_{i}": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True} for i in range(1, opti_settings["n_mast_segments"]+1)},
54             **{"t1_h_{i}": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True} for i in range(1, opti_settings["n_mast_segments"]+1)},
55             **{"d1_c_{i}": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True} for i in range(1, opti_settings["n_mast_segments"]+1)},
56         })
57     else:
58         var.update({
59             "d1": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True}, # Horizontal Brace Diameter [mm]
60             "t1": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True}, # Horizontal Brace Thickness [mm]
61             "d2": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True}, # Cross Brace Diameter [mm]
62             "t2": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True}, # Cross Brace Thickness [mm]
63         })
```

Main.py

```

56         **{"f"t1_c_{i}": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True} for
range(1, opti_settings["n_mast_segments"]+1)},          # Cross Brace Thickness      [mm]
57     })
58     else:
59         var.update({
60             **{"d1_{i}": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True} for i in
(1, opti_settings["n_mast_segments"]+1)},          # Brace Diameter      [mm]
61             **{"f"t1_{i}": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True} for
ange(1, opti_settings["n_mast_segments"]+1)},          # Brace Thickness      [mm]
62         })
63     else:
64         if opti_settings["brace_split"]:
65             var.update({
66                 "d1_h": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True},          # H
tal Brace Diameter      [mm]
67                 "t1_h": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True},          # H
tal Brace Thickness      [mm]
68                 "d1_c": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True},          # C
race Diameter      [mm]
69                 "t1_c": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True},          # C
race Thickness      [mm]
70             })
71         else:
72             var.update({
73                 "d1": {"value": brace_diameter, "bounds": brace_diameter_bounds, "active": True},          # Bra
meter      [mm]
74                 "t1": {"value": brace_thickness, "bounds": brace_thickness_bounds, "active": True},          # Bra
ckness      [mm]
75             })
76
77     # Static variables
78     Misc = {
79         "esize": 3,          # Element Size [mm]
80         "Hor_Force": 502.52,          # Horizontal Force (P_Load_z) [N]
81         "Ver_Force": -25.13E+3,          # Vertical Force (P_Load_y)      [N]
82         "f_y": 700 ,          # Column Yield Strength [MPa]
83         "f_y_brace": 235,          # Brace Yield Strength [MPa]
84         "E_mod": 200*1E3,          # Youngs Modulus [MPa]
85         "W_Force": -3.751E+3,          # Vertical Force COG (P_COG_y) [N]
86         "SW_filename": "LWC_L1_LINES.IGS",          # Filename for IGS File
87         "save_folder": "Optimization_Logs"          # Save Folder
88     }
89
90     # Solver Settings
91     Solver_Settings = {
92         "acc": 1e-3,          # Maximum objective function tolerance
93         "maxiter": 120,          # Maximum iterations
94         "Aggregate": "P-norm",          # None, "P-norm", "P-norm-mean", "KS", "KS_shift" (Write exacly)
95         "p_value": 8,          # Value for "P-norm" and "P-norm-mean"
96         "rho_value": 100,          # rho value used in KS
97         "relaxation": 0,          # Relaxation parameter used in aggregation
98         "use_acs": True,          # Toggle ACS to on (True) or off (False)
99     }
100
101     tic_lic = time.time()
102     # Launch MAPDL
103     mapdl = launch_mapdl(
104         run_location="Ansout",
105         log_apdl="apdl_log",
106         override=True,
107         nproc=7,
108         additional_switches="-p ansys -smp",

```

Main.py

```
109 )
110 toc_lic = time.time()
111 print(f"License opened in: {toc_lic-tic_lic:.2f} s")
112
113 # Ensures that MAPDL closes if something chrashes
114 try:
115     f1 = RunAPDL(mapdl,var,Misc,opti_settings)
116     result, txt_path, csv_path = optimization.run_optimization(mapdl, opti_settings, var, Misc, Solver_Setting)
117 finally:
118     mapdl.exit()
```